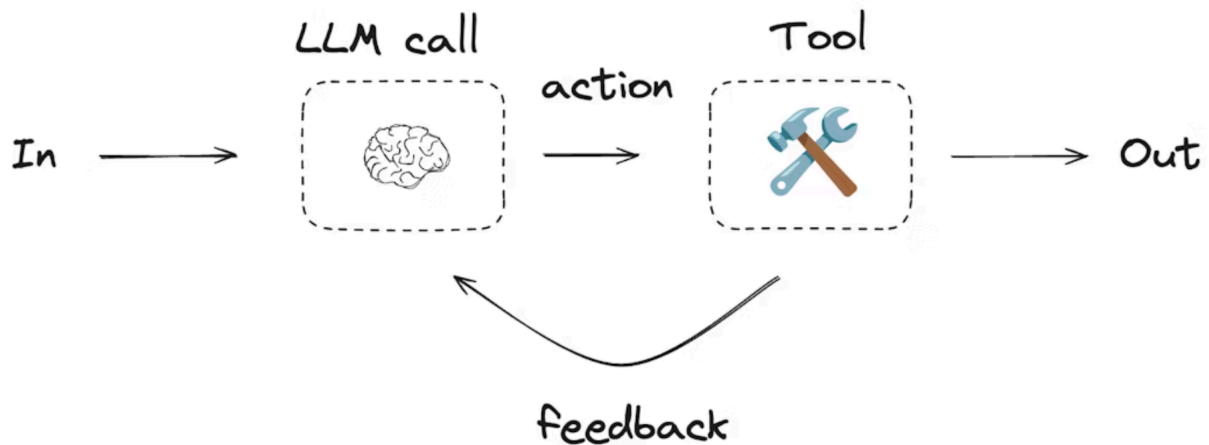


ReACT Agent



```
from langchain.tools import tool
```

```
# Define tools
```

```
@tool
```

```
def multiply(a: int, b: int) → int:
```

```
    """Multiply `a` and `b`.
```

```
    Args:
```

```
        a: First int
```

```
        b: Second int
```

```
    """
```

```
    return a * b
```

```
@tool
```

```
def add(a: int, b: int) → int:
```

```
    """Adds `a` and `b`.
```

```

Args:
    a: First int
    b: Second int
"""
return a + b

```

```

@tool
def divide(a: int, b: int) → float:
    """Divide `a` and `b`.
```

```

Args:
    a: First int
    b: Second int
"""
return a / b

```

```

# Augment the LLM with tools
tools = [add, multiply, divide]
tools_by_name = {tool.name: tool for tool in tools}
llm_with_tools = llm.bind_tools(tools)

```

```

from langgraph.graph import MessagesState
from langchain.messages import SystemMessage, HumanMessage, ToolMessage

```

```

# Nodes
def llm_call(state: MessagesState):
    """LLM decides whether to call a tool or not"""

    return {
        "messages": [
            llm_with_tools.invoke(

```

```

        [
            SystemMessage(
                content="You are a helpful assistant tasked with performing arithmetic on a set of inputs."
            )
        ]
        + state["messages"]
    )
]
}

```

```

def tool_node(state: dict):
    """Performs the tool call"""

    result = []
    for tool_call in state["messages"][-1].tool_calls:
        tool = tools_by_name[tool_call["name"]]
        observation = tool.invoke(tool_call["args"])
        result.append(ToolMessage(content=observation, tool_call_id=tool_call["id"]))
    return {"messages": result}

```

Conditional edge function to route to the tool node or end based upon whether the LLM made a tool call

```

def should_continue(state: MessagesState) → Literal["tool_node", END]:
    """Decide if we should continue the loop or stop based upon whether the LLM made a tool call"""

```

```

    messages = state["messages"]
    last_message = messages[-1]

```

```

    # If the LLM makes a tool call, then perform an action
    if last_message.tool_calls:
        return "tool_node"

```

```

# Otherwise, we stop (reply to the user)
return END

# Build workflow
agent_builder = StateGraph(MessagesState)

# Add nodes
agent_builder.add_node("llm_call", llm_call)
agent_builder.add_node("tool_node", tool_node)

# Add edges to connect nodes
agent_builder.add_edge(START, "llm_call")
agent_builder.add_conditional_edges(
    "llm_call",
    should_continue,
    ["tool_node", END]
)
agent_builder.add_edge("tool_node", "llm_call")

# Compile the agent
agent = agent_builder.compile()

# Show the agent
display(Image(agent.get_graph(xray=True).draw_mermaid_png()))

# Invoke
messages = [HumanMessage(content="Add 3 and 4.")]
messages = agent.invoke({"messages": messages})
for m in messages["messages"]:
    m.pretty_print()

```